# Repository

Repository · Persist users behind collection-like API · Build with Waqar

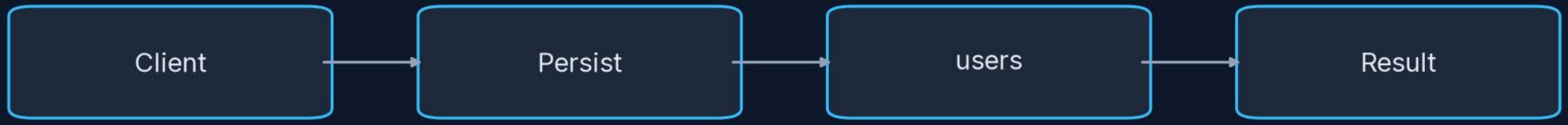
# Today's Hook

- Problem: Persist users behind collection-like API
- Pattern: Repository — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Persist users behind collection-like API
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Repository: Persist users behind collection-like API

# Real-World Scenario

- Running example: Persist users behind collection-like API
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.enterprise.repository.problem;

import java.util.*;

public final class NaiveUsers {
  private final Map<String,String> db=new HashMap<>();
  public void insert(String id,String email){ db.put(id,email); }
  public String get(String id){ return db.get(id); } // null
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```java
package com.buildwithwaqar.patterns.enterprise.repository.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.concurrent.locks.*;
import java.util.function.*;

record User(String id, String email){}
interface UserRepository {
  void save(User u); Optional<User> findById(String id); List<User> findAll();
}
final class InMemoryUserRepository implements UserRepository {
  private final Map<String,User> db=new LinkedHashMap<>();
  public void save(User u){ db.put(u.id(), u); }
  public Optional<User> findById(String id){ return Optional.ofNullable(db.get(id)); }
  public List<User> findAll(){ return List.copyOf(db.values()); }
}
```

# Runnable main

```
package com.buildwithwaqar.patterns.enterprise.repository.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Repository: users collection API ===");
    System.out.println("--- naive ---");
var n=new NaiveUsers(); n.insert("1","a@b.c"); System.out.println(n.get("missing"));
System.out.println("--- repository ---");
UserRepository repo=new InMemoryUserRepository(); repo.save(new User("1","a@b.c"));
System.out.println(repo.findById("1")); System.out.println(repo.findById("x"));
  }
}
```

# Walkthrough Output

```
=== Repository: users collection API ===
--- naive ---
null
--- repository ---
Optional[User[id=1, email=a@b.c]]
Optional.empty
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Repository.

# Related Patterns

- Contrast Repository with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Repository → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.